

Phase 0 - Project Definition

- ~~[] Define the business problem~~
- ~~[] Define project scope~~
- ~~[] Define out-of-scope items~~
- ~~[] Define target users~~
- ~~[] Define business questions the platform should answer~~
- ~~[] Define success criteria~~
- ~~[] Select primary FAOSTAT datasets/domains~~
- ~~[] Decide initial historical period~~
- ~~[] Create project README~~
- ~~[] Create project glossary~~

Deliverable: Project Charter

Phase 1 - Local Development & Git Setup

- ~~[] Create local project directory~~
- ~~[] Initialize Git repository~~
- ~~[] Create GitHub repository~~
- ~~[] Create production-style folder structure~~
- ~~[] Create `.gitignore`~~
- ~~[] Create Python virtual environment~~
- ~~[] Create dependency file~~
- ~~[] Configure VS Code~~
- ~~[] Define branch strategy~~
- ~~[] Define commit message convention~~
- ~~[] Add initial README~~
- ~~[] Push initial repository~~

Deliverable: Clean development workspace and GitHub repository

Phase 2 - Data Discovery & Source Profiling

FAOSTAT

- ~~[] Study FAOSTAT API documentation~~
- ~~[] Identify relevant domains~~
- ~~[] Understand dimensions~~
- ~~[] Understand indicator definitions~~
- ~~[] Understand country/area codes~~
- ~~[] Understand year granularity~~
- ~~[] Investigate missing values~~
- ~~[] Investigate revisions~~

- ~~[] Inspect API pagination~~
- ~~[] Inspect API response formats~~
- ~~[] Pull a small sample locally~~
- ~~[] Profile sample JSON/CSV~~
- ~~[] Document source quirks~~

World Bank

- ~~[] Identify possible enrichment indicators~~
- ~~[] Understand World Bank country identifiers~~
- ~~[] Investigate mapping with FAOSTAT countries~~
- ~~[] Document potential integration strategy~~

Deliverable: Source Analysis Document + Sample Data

Phase 3 - Architecture & Data Contracts

- ~~[] Draw logical architecture~~
- ~~[] Draw physical architecture~~
- ~~[] Define source-to-target flow~~
- ~~[] Define RAW responsibilities~~
- ~~[] Define CLEAN responsibilities~~
- ~~[] Define CONSUMPTION responsibilities~~
- ~~[] Define PUBLISH responsibilities~~
- ~~[] Define ingestion metadata~~
- ~~[] Define naming conventions~~
- ~~[] Define schema conventions~~
- ~~[] Define data types~~
- ~~[] Define data contracts~~
- ~~[] Define expected refresh frequency~~
- ~~[] Define retry behaviour~~
- ~~[] Define duplicate handling strategy~~
- ~~[] Define schema evolution strategy~~
- ~~[] Create ADR-001 for major architecture decisions~~

Deliverable: Architecture v1

Phase 4 - Snowflake Platform Foundation

- ~~[] Design Snowflake databases~~
- ~~[] Design schemas~~
- ~~[] Design warehouses~~
- ~~[] Define warehouse sizing assumptions~~
- ~~[] Configure auto-suspend~~
- ~~[] Configure auto-resume~~

- ~~[] Define Snowflake roles~~
- ~~[] Define service roles~~
- ~~[] Define developer role~~
- ~~[] Define analyst/consumer role~~
- ~~[] Create base Snowflake objects~~
- ~~[] Validate permissions~~

Deliverable: Snowflake platform skeleton

Phase 5 - AWS Landing Zone

- ~~[] Create AWS S3 bucket~~
- ~~[] Design bucket prefix structure~~
- ~~[] Define raw file naming convention~~
- ~~[] Understand IAM users vs roles~~
- ~~[] Create Snowflake-compatible IAM permissions~~
- ~~[] Configure Snowflake Storage Integration~~
- ~~[] Create external stage~~
- ~~[] Validate Snowflake → S3 access~~
- ~~[] Define retention/lifecycle approach~~
- ~~[] Ensure credentials are never committed to Git~~

Example logical structure:

```
source/domain/year/month/day/run_id/file
```

Deliverable: Secure AWS S3 → Snowflake connection

Phase 6 - Source Ingestion

- ~~[] Build FAOSTAT extraction module~~
- ~~[] Handle pagination~~
- ~~[] Handle HTTP errors~~
- ~~[] Handle retries~~
- ~~[] Handle timeouts~~
- ~~[] Capture request metadata~~
- ~~[] Generate ingestion/run ID~~
- ~~[] Capture extraction timestamp~~
- ~~[] Write source files without modifying them~~
- ~~[] Upload files to S3~~
- ~~[] Make ingestion idempotent~~
- ~~[] Log row/file counts~~
- ~~[] Test partial failures~~
- ~~[] Test repeated executions~~

Deliverable: Re-runnable FAOSTAT → S3 ingestion pipeline

Phase 7 - RAW / Landing Layer

- ~~[] Create appropriate file formats~~
- ~~[] Create RAW tables~~
- ~~[] Load files from S3~~
- ~~[] Preserve original source attributes~~
- ~~[] Capture filename~~
- ~~[] Capture ingestion timestamp~~
- ~~[] Capture run ID~~
- ~~[] Capture source system metadata~~
- ~~[] Prevent duplicate loads~~
- ~~[] Create load auditing~~
- ~~[] Validate source vs target counts~~

Deliverable: Auditable immutable RAW layer

Phase 8 - CLEAN / Standardization Layer

- ~~[] Parse source structures~~
- ~~[] Flatten nested data where required~~
- ~~[] Normalize column names~~
- ~~[] Standardize country identifiers~~
- ~~[] Standardize indicator identifiers~~
- ~~[] Standardize years/dates~~
- ~~[] Standardize units~~
- ~~[] Handle null values~~
- ~~[] Handle duplicates~~
- ~~[] Handle malformed records~~
- ~~[] Implement data quality rules~~
- ~~[] Separate rejected records where appropriate~~
- ~~[] Document transformation rules~~

Deliverable: Trusted standardized datasets

Phase 9 - World Bank Enrichment

- ~~[] Select useful economic/demographic indicators~~
- ~~[] Build World Bank API ingestion~~
- ~~[] Land World Bank raw data in S3~~
- ~~[] Load RAW data into Snowflake~~
- ~~[] Standardize World Bank data~~

- ~~[] Build country crosswalk~~
- ~~[] Handle unmatched countries~~
- ~~[] Integrate World Bank and FAOSTAT data~~

Deliverable: Multi-source integrated dataset

Phase 10 - Dimensional Modeling

- ~~[] Identify business grain~~
- ~~[] Design fact tables~~
- ~~[] Design country dimension~~
- ~~[] Design indicator dimension~~
- ~~[] Design date/year dimension~~
- ~~[] Design geography/region dimension~~
- ~~[] Consider source dimension~~
- ~~[] Determine surrogate-key strategy~~
- ~~[] Determine SCD requirements~~
- ~~[] Define relationships~~
- ~~[] Validate model against business questions~~
- ~~[] Create ER/data-model diagram~~

Deliverable: Consumption-layer dimensional model

Phase 11 - Data Quality Framework

- ~~[] Test uniqueness~~
- ~~[] Test mandatory fields~~
- ~~[] Test accepted values~~
- ~~[] Test relationships~~
- ~~[] Test year ranges~~
- ~~[] Test numerical ranges~~
- ~~[] Test duplicate detection~~
- ~~[] Test referential integrity~~
- ~~[] Validate source-vs-target counts~~
- ~~[] Track rejected records~~
- ~~[] Store quality test results~~
- ~~[] Decide pipeline behaviour on failed checks~~

Deliverable: Automated data-quality suite

Phase 12 - Transformation & Orchestration

- ~~[] Understand Streams~~
- ~~[] Understand Tasks~~

- ~~[] Understand Dynamic Tables~~
- ~~[] Compare appropriate approaches~~
- ~~[] Select orchestration design~~
- ~~[] Implement incremental processing~~
- ~~[] Configure dependencies~~
- ~~[] Configure schedules~~
- ~~[] Configure retries where applicable~~
- ~~[] Test reruns~~
- ~~[] Test missed schedules~~
- ~~[] Test duplicate source arrival~~

Deliverable: Automated Snowflake processing pipeline

Phase 13 - Security & Governance

- ~~[] Implement RBAC~~
- ~~[] Apply least-privilege access~~
- ~~[] Separate service and human roles~~
- ~~[] Restrict RAW access~~
- ~~[] Review ownership~~
- ~~[] Review future sensitive data handling~~
- ~~[] Configure appropriate grants~~
- ~~[] Document access model~~
- ~~[] Test unauthorized access scenarios~~

Deliverable: Production-style security model

Phase 14 - Monitoring & Cost Management

- ~~[] Monitor warehouse usage~~
- ~~[] Monitor query history~~
- ~~[] Monitor pipeline runs~~
- ~~[] Monitor failed loads~~
- ~~[] Monitor data freshness~~
- ~~[] Monitor data quality failures~~
- ~~[] Track processed records~~
- ~~[] Track processing duration~~
- ~~[] Examine Snowflake credit usage~~
- ~~[] Tune warehouse configuration~~
- ~~[] Detect expensive queries~~
- ~~[] Create operational queries/dashboard~~

Deliverable: Observability + cost-control layer

Phase 15 - Consumption & Streamlit

- ☐ ~~Define dashboard personas~~
- ☐ ~~Define KPIs~~
- ☐ ~~Build analytical views~~
- ☐ ~~Build food security trends~~
- ☐ ~~Build country comparisons~~
- ☐ ~~Build regional comparisons~~
- ☐ ~~Add economic/demographic enrichment~~
- ☐ ~~Build Streamlit application~~
- ☐ ~~Add filters~~
- ☐ ~~Validate calculations~~
- ☐ ~~Review query performance~~

Deliverable: Global Food Security Intelligence application

Phase 16 - CI/CD

- ☐ Create GitHub Actions workflow
- ☐ Protect credentials with GitHub Secrets
- ☐ Add Python checks
- ☐ Add SQL validation
- ☐ Add automated tests
- ☐ Create development deployment workflow
- ☐ Define promotion strategy
- ☐ Understand DEV → TEST → PROD deployment concepts
- ☐ Prevent accidental production deployment

Deliverable: Repeatable deployment process

Phase 17 - Production Failure Drills

Intentionally break things.

- ☐ Duplicate source file
- ☐ Missing file
- ☐ Bad JSON/CSV
- ☐ API timeout
- ☐ HTTP failure
- ☐ New source column
- ☐ Missing source column
- ☐ Invalid country code
- ☐ Pipeline interruption halfway through
- ☐ Rerun historical period

- ☐ Incorrect transformation
- ☐ Warehouse accidentally left running
- ☐ Unauthorized-user test

For each incident:

- ☐ Detect
- ☐ Diagnose
- ☐ Recover
- ☐ Prevent recurrence
- ☐ Document resolution

Deliverable: Incident/Recovery Runbook

Phase 18 - Final Production Review

- ☐ Review architecture
- ☐ Review repository
- ☐ Review SQL quality
- ☐ Review Python quality
- ☐ Review security
- ☐ Review costs
- ☐ Review scalability
- ☐ Review idempotency
- ☐ Review observability
- ☐ Review documentation
- ☐ Review business usefulness
- ☐ Identify technical debt
- ☐ Produce architecture diagram v2
- ☐ Complete final README
- ☐ Produce project retrospective

Final Deliverable: Production-style Snowflake + AWS data engineering portfolio project